Programiranje za web
Predavanje 2: Modeliranje podataka, ORM i migracije

doc. dr. sc. Tomislav Petković mag. ing. Josip Torić

15.10.2026.





Sveučilište u Zadru
Universitas Studiorum
Jadertina | 1396 | 2002 |

Plan predavanja

1. Baze podataka: vrste, struktura, povezivanje zapisa
2. Ograničenja i transakcije
3. Objektno-relacijsko preslikavanje
4. Migracije i početni podatci

Na kraju predavanja baza ima tablice `clubs` i `users`, migracija je zapisana u repozitoriju, a početni podatci se stvaraju jednom naredbom.

Ishodi učenja

Nakon ovog predavanja student može:

- razdvojiti pojmove domene u tablice i povezati ih stranim ključem
- obrazložiti izbor relacijske baze i izbor primarnog ključa
- objasniti zašto ograničenje stoji u shemi baze, a ne samo u kodu
- napisati model u SQLAlchemyju i razlikovati ga od sheme API-ja
- promijeniti shemu migracijom i objasniti doseg naredbe `downgrade`

Domena primjera

Sustav prijava klubova na powerlifting natjecanje. Danas su potrebna samo dva pojma:

- **Klub** ima naziv i grad.
- **Korisnik** je račun za prijavu u sustav i nosi ulogu. Uloga je `admin` ili `club`. Račun s ulogom `admin` je organizator natjecanja i ne pripada nijednom klubu; račun s ulogom `club` pripada točno jednom klubu, a svaki klub ima jedan takav račun.

Natjecatelji, natjecanja i prijave dolaze poslije. Prvo se postavlja temelj na kojem počivaju prijava u sustav i pravo pristupa.

Baze podataka

Pohrana izvan aplikacije, vrste baza, struktura relacijske baze, povezivanje zapisa i ključevi.

Pohrana izvan aplikacije

Ruta iz prethodnog predavanja vraćala je popis zapisan u kodu:

```
@router.get("/competitions")
def list_competitions():
   return [{"id": 7, "naziv": "Zimski kup"}]
```

Ovakvo zapisivanje koristi samo memoriju procesa pa:

- novi zapis traži izmjenu koda i ponovno pokretanje
- ponovno pokretanje uklanja sve što je nastalo u radu
- dva istodobno pokrenuta primjerka aplikacije imaju različit sadržaj

Podatci zato stoje u zasebnom sustavu, koji ih pohranjuje na disk i poslužuje svim primjercima aplikacije. Takav se sustav naziva sustav za upravljanje bazom podataka.

Relacijske i nerelacijske baze

Sustavi za upravljanje bazama dijele se prema tome kako organiziraju zapise.

Vrsta	Organizacija zapisa	Primjer sustava
relacijska	tablice sa zadanom shemom	PostgreSQL
dokumentna	dokumenti bez zadane sheme	MongoDB
ključ i vrijednost	vrijednost po ključu	Redis

U ovom kolegiju koristi se relacijska baza jer domena ima nepromjenjiv oblik zapisa, veze među njima i pravila koja se moraju provoditi nad svakim upisom. Dokumentne baze bolje podnose zapise različita oblika, a pravila o vezama ostavljaju aplikaciji.

Struktura relacijske baze

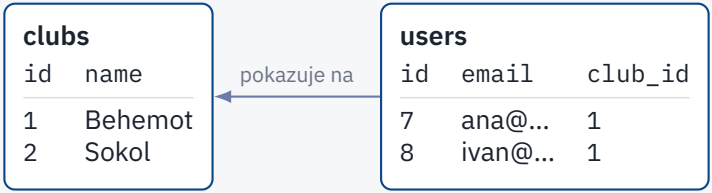
Relacijska baza podatke drži u tablicama. Preslikavanje pojmova domene je izravno:

Pojam domene	U bazi
vrsta zapisa	tablica `clubs`
jedan zapis	redak
svojstvo zapisa	stupac `city`

Tablica se izvodi iz pojma koji u domeni postoji i traje, a ne iz obrasca na zaslonu. Obrazac se mijenja češće nego što se mijenja to da klub ima naziv i grad.

Povezivanje zapisa

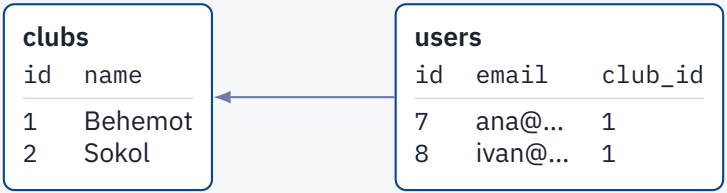
Korisnik pripada klubu. Tu vezu treba zapisati tako da je baza može provjeriti.



Stupac `club_id` sadrži vrijednost stupca `id` iz tablice `clubs` i naziva se strani ključ. Baza odbija upis retka čiji `club_id` ne postoji, kao i brisanje kluba na koji se neki redak poziva.

Primarni ključ

Da bi jedan redak mogao pokazivati na drugi, redak treba oznaku koja ga jednoznačno određuje. Ta se oznaka naziva primarni ključ.



Veza je time opisana dvama ključevima: primarnim u tablici na koju se pokazuje i stranim u tablici koja pokazuje.

Izbor primarnog ključa

Za primarni ključ postoje dva izbora:

- **podatak iz domene**, primjerice adresa e-pošte korisnika
- **vrijednost bez značenja**, stupac `id` koji baza sama dodjeljuje

Podatak iz domene podložan je promjeni. Promijeni li korisnik adresu e-pošte, a na njegov se redak poziva pet drugih tablica, sve te vrijednosti treba promijeniti u istoj transakciji; propust ostavlja zapise koji pokazuju na redak kojega nema.

Vrijednost bez značenja nema razloga za promjenu, pa taj slučaj ne nastaje. Jedinstvenost podatka iz domene i dalje se čuva, ali ograničenjem `UNIQUE`, a ne time što je taj podatak ključ.

Operacije nad bazom

Naredbe koje baza prima dijele se u tri skupine prema tome na što se odnose:

Skupina	**Naredbe**	**Predmet**
definicija sheme	`CREATE`, `ALTER`, `DROP`	oblik tablica
rad s podatcima	`SELECT`, `INSERT`, `UPDATE`, `DELETE`	sadržaj tablica
upravljanje transakcijom	`BEGIN`, `COMMIT`, `ROLLBACK`	početak i kraj niza izmjena

Aplikacija u radu koristi drugu i treću skupinu. Prva se izvodi pri postavljanju baze i pri svakoj kasnijoj promjeni oblika podataka.

Vrijeme kao podatak

Rokovi, trenutci prijave i vremena izmjene zapisuju se kao datum i vrijeme. Njih možemo zapisati kao:

Tip stupca	Što je zapisano	Primjer vrijednosti
`timestamp`	datum i vrijeme, bez zone	`2027-02-14 23:59:00`
`timestamptz`	trenutak na vremenskoj osi	`2027-02-14 23:59:00+01`

`timestamp` ne određuje trenutak jednoznačno. Ponoć u Zadru i ponoć u Londonu zapisuju se jednako, a razlikuju se za sat. `timestamptz` nosi odmak od koordiniranoga svjetskog vremena (UTC), pa je trenutak jednoznačan bez obzira na to tko zapis čita i gdje.

Tko provodi pravilo o zoni

PostgreSQL vrijednost u stupcu `timestamptz` pohranjuje kao trenutak u UTC-u, bez zone. Pri čitanju je prikazuje u zoni koju sesija traži postavkom `TimeZone`; nije li postavljena, vrijedi zadana zona poslužitelja. Pretvorba u lokalno vrijeme time je posao prikaza, a ne pohrane.

Pritom je potrebno biti oprezan: **baza ne odbija vrijednost bez oznake zone**. Tumači je u istoj toj zoni i upis prolazi. Zahtjev da vrijednost nosi oznaku zone provodi se u aplikaciji, shemom na ulazu, jednako kao i svako drugo pravilo o obliku podataka.

Ograničenja i transakcije

Uvjeti nad podatcima, istodobni zahtjevi, atomarnost, indeksi.

Ograničenja

Ograničenje je uvjet zapisan u shemi tablice. Sustav ga provjerava pri svakom upisu i svakoj izmjeni.

Ograničenje	**Uvjet**	**Primjer**
`NOT NULL`	vrijednost postoji	naziv kluba
`UNIQUE`	nema dvaju jednakih	adresa e-pošte
`FOREIGN KEY`	vrijednost postoji u drugoj tablici	`club_id`
`CHECK`	vrijednost zadovoljava izraz	masa veća od nule

Uvjet se provjerava neovisno o izvoru izmjene: pri upisu iz aplikacije, pri ručnom upisu iz konzole i pri uvozu iz datoteke.

Istodobni zahtjevi i jedinstvenost

Uobičajen redoslijed u kodu je: provjeri postoji li već korisnik s tom adresom, pa upiši ako ne postoji. Dva istodobna zahtjeva tada mogu proći oba:

Trenutak	Zahtjev A	Zahtjev B
1	provjera: adresa je slobodna	
2		provjera: adresa je slobodna
3	upis prolazi	
4		upis prolazi

Nastaju dva retka s istom adresom, a prijava u sustav više nije jednoznačna. Nijedan zahtjev nije pogrešno napisan: razmak između provjere i upisa dovoljan je da se stanje promijeni.

Uz ograničenje `UNIQUE` četvrti upis ne prolazi, a aplikacija tu pogrešku pretvara u odgovor `409`.

Nepotpun niz izmjena

Početno punjenje baze upisuje tri retka: klub, račun organizatora i račun toga kluba.

Ne prođe li treći upis, a prva dva su već potvrđena, u bazi ostaju klub i račun organizatora, bez računa kluba. To stanje nije predvidio ni kod koji upisuje ni kod koji čita: postoji klub kojim se nitko ne može služiti, a ponovno pokretanje punjenja zatječe polovicu posla obavljenom.

Isto vrijedi za svaku operaciju koja mijenja više redaka. Prijava natjecatelja upisuje prijavu i umanjuje broj slobodnih mjesta; prođe li samo prvo, broj mjesta više ne odgovara broju prijava.

Transakcija

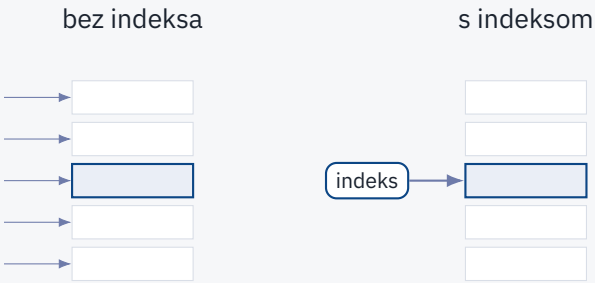
Transakcija je niz operacija koje sustav izvodi kao jednu: ili sve budu potvrđene, ili nijedna ne ostavi traga. To se svojstvo naziva **atomarnost**.

- Niz počinje naredbom `BEGIN`, a završava potvrdom (`COMMIT`) ili poništavanjem (`ROLLBACK`).
- Do potvrde promjene su vidljive samo unutar te transakcije.
- Pri pogrešci sustav vraća stanje na ono prije naredbe `BEGIN`, bez obzira na to koliko je upisa dotad prošlo.

Time nepotpun niz izmjena s prethodnog slajda ne može nastati: ili su sva tri retka u bazi, ili nijedan.

Indeks

Bez indeksa sustav pri pretrazi po stupcu čita svaki redak tablice i uspoređuje vrijednost. Indeks je zasebna struktura koja traženi redak pronalazi bez tog čitanja.



Primarni ključ i ograničenje `UNIQUE` indeks stvaraju sami, jer bez njega provjera jedinstvenosti ne bi bila izvediva u razumnom vremenu.

Objektno-relacijsko preslikavanje

Uloga ORM-a, modeli, odnos modela i sheme, veza s bazom i sesija.

Uloga ORM-a

Baza prima SQL. Aplikacija radi s objektima. ORM (engl. *object-relational mapping*) preslikava jedno u drugo: klasu u tablicu, primjerak klase u redak, svojstvo u stupac.

```
SELECT id, name, city FROM clubs WHERE city = 'Zadar';
```

```
result = await session.execute(
   select(Club).where(Club.city == "Zadar")
)
clubs = result.scalars().all()
```

Drugi zapis proizvodi prvi. ORM ne uklanja potrebu za poznavanjem SQL-a: kad je upit spor ili vraća neočekivan rezultat, odgovor se traži u SQL-u koji je nastao.

Model: `Club`

```python
# app/models/club.py

class Club(Base):
    __tablename__ = "clubs"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), unique=True, nullable=
        False)
    city: Mapped[str] = mapped_column(String(80), nullable=False)

    users: Mapped[list[User]] = relationship(back_populates="club")
```

Prva tri retka opisuju stupce i ograničenja koja ulaze u shemu tablice. Posljednji redak ne stvara stupac `users`: on ORM-u govori da su korisnici toga kluba oni retci tablice `users` kojima `club_id` odgovara ovom `id`. Zahvaljujući tomu se umjesto pisanja upita čita `club.users`, a upit sastavlja ORM.

Model: User

```python
# app/models/user.py
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, nullable=
        False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(20), nullable=False)

    club_id: Mapped[Optional[int]] = mapped_column(
       ForeignKey("clubs.id"), nullable=True
    )
    club: Mapped[Optional[Club]] = relationship(back_populates="users")
```

Model i shema

Dva su opisa istog pojma, s različitim zadaćama:

- **model** u `models`: kako zapis izgleda u bazi
- **shema** u `schemas`: kako zapis putuje kroz API

Razdvojeni su jer se ne poklapaju:

- `password_hash` postoji u bazi, a ne izlazi ni u jednom odgovoru
- pri stvaranju korisnika lozinka ulazi, pri čitanju ne izlazi, pa ulazna i izlazna shema nisu iste
- `id` dodjeljuje baza, pa ga klijent ne šalje, ali ga dobiva natrag

Da su model i shema jedno te isto, dodavanje stupca za internu uporabu, primjerice broja neuspjelih prijava, istoga bi trena promijenilo sadržaj svakog odgovora prema klijentu.

Skup dopuštenih vrijednosti

Stupac `role` u modelu je niz znakova, a u domeni ima samo dvije dopuštene vrijednosti, `admin` i `club`. Sama shema tablice pritom ne odbija upis vrijednosti `admn`.

Posljedica nije poruka o pogrešci nego tiha promjena ponašanja. Provjera koja traži ulogu `admin` za takav redak ne prolazi, a ne prolazi ni ona koja traži `club`. Korisnik postoji, prijava u sustav uspijeva, a nijedna mu ruta nije dostupna; uzrok se ne vidi ni u jednoj poruci.

Skup se može ograničiti na tri mjesta, a izbor mjesta određuje i tko je obuhvaćen i koliko stoji promjena popisa.

Prvo mjesto: tip stupca u bazi

```
role: Mapped[str] = mapped_column(
   Enum("admin", "club", name="user_role"), nullable=False
)
```

Gdje: u shemi tablice, opisano u sloju `models` i uneseno migracijom.

Kada se provjerava: pri svakom upisu i izmjeni, u sustavu za upravljanje bazom.

Vrijednost izvan popisa odbija se bez obzira na to odakle dolazi. Dodavanje treće uloge traži novu migraciju, jer se mijenja tip u shemi.

Drugo mjesto: uvjet nad tablicom

```
# app/models/user.py
__table_args__ = (
   CheckConstraint("role IN ('admin', 'club')", name="ck_user_role"),
)
```

Gdje: također u shemi tablice, kao ograničenje `CHECK`.

Kada se provjerava: pri svakom upisu i izmjeni, jednako kao tip stupca.

Razlika prema prvom mjestu je u obliku, ne u dosegu: `CHECK` prima proizvoljan izraz, pa istim sredstvom pokriva i uvjete koje popis vrijednosti ne može izraziti. I ovdje promjena popisa traži migraciju.

Ovo je izbor ovoga projekta: stupac je `String(20)`, a skup vrijednosti provodi `CHECK`.

Treće mjesto: shema na ulazu u aplikaciju

```python
class UserCreate(BaseModel):
   role: Literal["admin", "club"]
```

Gdje: u sloju `schemas`, izvan baze.

Kada se provjerava: pri razlaganju tijela zahtjeva, prije nego što funkcija rute počne raditi.

Odgovor imenuje polje koje nije prošlo, a promjena popisa ne traži migraciju. Zauzvrat, upis koji ne dolazi kroz aplikaciju, primjerice iz konzole baze, ovim uvjetom nije obuhvaćen.

Veza s bazom i sesija

```python
# app/core/database.py
engine = create_async_engine(settings.DATABASE_URL, pool_pre_ping=True)
AsyncSessionLocal = async_sessionmaker(bind=engine, class_=AsyncSession)

# app/core/deps.py
async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

`engine` drži skup otvorenih veza prema bazi. `session` je jedna cjelina rada: prati promijenjene objekte i na kraju ih zajedno potvrđuje ili poništava.

Jedna sesija po zahtjevu

Sesija nije samo veza prema bazi. Unutar nje teče jedna transakcija i ona prikuplja sve promjene koje zahtjev napravi.

Zato je dva zahtjeva ne dijele. Kad bi je dijelila:

- potvrda jednoga potvrdila bi i nedovršene promjene drugoga
- pogreška u jednome poništila bi i rad drugoga

Sesiju zato otvara ovisnost `get_db` pri ulasku u rutu i zatvara je pri izlasku. Svaki zahtjev dobiva svoju.

Čekanje na bazu

Upit prema bazi traje, a proces za to vrijeme nema što raditi. Dva su pristupa tom čekanju:

- **sinkrono**: dretva koja je poslala upit čeka odgovor i u tom razdoblju ne poslužuje druge zahtjeve; istodobnost se postiže većim brojem dretvi ili procesa
- **asinkrono**: dretva za vrijeme čekanja preuzima drugi zahtjev i vraća se kad odgovor stigne

Pristup mora biti dosljedan kroz cijeli sloj pristupa bazi: asinkroni kod traži upravljački program koji podržava takav rad. Adresa za spajanje ga i imenuje, pa `postgresql+asyncpg://` označava asinkroni, a `postgresql://` obični.

U ovom se projektu radi asinkrono, pa su i funkcije koje dodiruju bazu označene s `async`, a njihovi pozivi s `await`.

Sinkrono i asinkrono

	sinkrono	**asinkrono**
kod	bez `async` i `await`	svaka funkcija do baze nosi `async` i `await`
istodobnost	jedna dretva po zahtjevu	jedna dretva poslužuje mnogo zahtjeva koji čekaju
knjižnice	gotovo svaka radi	treba asinkronu inačicu, sinkrona blokira
cijena	dretva troši memoriju, pa mnogo zahtjeva znači mnogo dretvi	jedan sinkroni poziv zaustavlja sve ostale zahtjeve te dretve
isplati se	kad je posao kratak ili ga je malo	kad se većinu vremena čeka mreža ili baza

Migracije i početni podatci

Razilaženje modela i baze, sadržaj migracije, naredbe, granice automatskog stvaranja, punjenje baze.

Model i tablica se razilaze

Model mijenjamo u Python kodu, ali se te promjene ne primjenjuju na bazu same od sebe. Dodamo li stupac u model, tablica ostaje kakva je bila.

Od tog trenutka model i tablica opisuju različite stvari. Prvi upit koji novi stupac spominje ne prolazi, a pogreška dolazi iz baze, daleko od retka koji je stupac dodao.

Najjednostavnije rješenje je obrisati tablice i stvoriti ih iznova iz modela. To je bezopasno dok je baza prazna. Čim u njoj ima podataka, briše i njih.

Migracija

Potreban je zapis koji staru shemu prevodi u novu, bez gubitka sadržaja. Takav se zapis naziva migracija: datoteka u repozitoriju koja opisuje jednu promjenu sheme.

To osigurava da:

- svaki član tima izvodi iste promjene istim redoslijedom
- produkcijska baza prolazi kroz iste korake kao razvojna
- promjena sheme ostaje vidljiva u povijesti, uz kod koji ju je tražio

Sadržaj migracije

```
# alembic/versions/001_init_clubs_and_users.py
revision = "001"
down_revision = None
def upgrade() -> None:
   op.create_table(
      "clubs",
      sa.Column("id", sa.Integer(), primary_key=True),
      sa.Column("name", sa.String(length=120), nullable=False),
      sa.Column("city", sa.String(length=80), nullable=False),
      sa.UniqueConstraint("name"),
   )
def downgrade() -> None:
   op.drop_table("clubs")
```

`revision` imenuje ovu promjenu, `down_revision` onu prije nje; iz toga slijedi redoslijed. Baza pamti do koje je migracije došla, pa svaka prolazi točno jednom.

Kako nastaje migracija

Migracija se ne piše od nule nego nastaje iz razlike između modela i baze:

1. U modelu se napravi promjena, primjerice doda stupac.
2. `alembic revision --autogenerate` usporedi modele s trenutnim stanjem baze i razliku zapiše u novu datoteku u `alembic/versions/`.
3. Datoteka se pročita i po potrebi ispravi.
4. `alembic upgrade head` je primjenjuje.

Treći korak nije formalnost. Usporedba ne pogađa svaku namjeru, pa nastala datoteka zna opisivati nešto drugo od onoga što je autor htio.

Nastala datoteka ulazi u isti commit kao i promjena modela zbog koje je nastala.

Naredbe

```
# create a migration from the difference between models and database
alembic revision --autogenerate -m "init clubs and users"

# apply everything not yet applied
alembic upgrade head

# undo the last migration
alembic downgrade -1

# which migration is the database on
alembic current
```

`upgrade head` primjenjuje sve migracije kroz koje baza još nije prošla, redoslijedom koji slijedi iz njihovih oznaka.

Granice automatskog stvaranja

`autogenerate` uspoređuje modele s trenutnim stanjem baze i zapisuje razliku. Za neke promjene ta usporedba nije dostatna:

- Promjena naziva stupca vidi se kao nestanak jednog i pojava drugog, pa nastaju brisanje i stvaranje.
- Promjena tipa stupca traži i pravilo po kojem se postojeće vrijednosti pretvaraju, a ono iz usporedbe ne slijedi.
- Novi stupac s oznakom `NOT NULL` nad tablicom koja već ima retke ne može se dodati bez zadane vrijednosti, jer bi postojeći retci ostali bez vrijednosti. Nad praznom tablicom isti upis prolazi.

Prva dva slučaja ne javljaju pogrešku. Migracija se izvede do kraja i `upgrade` prođe uredno; nedostaju samo podatci.

Doseg naredbe `downgrade` pritom je ograničen: brisanje stupca ne može se poništiti jer podatci koje je sadržavao više ne postoje.

Čitanje migracije prije pokretanja

Nakon preimenovanja stupca `email` u `username`, `autogenerate` zapisuje ovo:

```
def upgrade() -> None:
   op.drop_column("users", "email")
   op.add_column("users", sa.Column("username", sa.String(255)))
```

Znak za uzbunu je par `drop_column` i `add_column` nad istom tablicom u istoj migraciji: to je preimenovanje koje je izgubilo sadržaj. Ispravak je jedan redak:

```
def upgrade() -> None:
   op.alter_column("users", "email", new_column_name="username")
```

Zato se svaka nastala migracija pročita prije pokretanja. U datoteci je ispravak trenutačan; nakon naredbe `upgrade` nad produkcijskom bazom sadržaja stupca više nema.

Dodavanje indeksa

Indeks se traži uz stupac u modelu, a u shemu ulazi migracijom:

```
# in the model
club_id: Mapped[int] = mapped_column(ForeignKey("clubs.id"), index=True)

# in the migration
op.create_index("ix_users_club_id", "users", ["club_id"])
```

Indeks ubrzava pretragu, spajanje tablica i sortiranje po tom stupcu. Zauzima prostor i usporava upis, jer se pri svakoj izmjeni i sam mijenja.

Konvencija

Indeks se dodaje na stupce po kojima se često filtrira, tipično na strane ključeve. Indeksiranje svakog stupca unaprijed plaća se pri svakom upisu, a rijetko donosi korist.

Početni podatci

Migracija mijenja oblik tablica, ne njihov sadržaj. Za isprobavanje su potrebni i zapisi: organizator, klub i račun toga kluba.

Ručni upis ima dvije mane:

- svaki član tima ima svoje nazive, lozinke i oznake, pa ono što radi kod jednoga ne radi kod drugoga
- zapisi nestaju pri svakom brisanju baze, a baza se briše često: pri isprobavanju migracija i pri prelasku na drugu granu

Zato upis ide naredbom koja se smije pokrenuti bilo koliko puta i svaki put dati isto stanje. Taj se postupak naziva punjenje početnim podatcima (engl. *seeding*).

Ponovljivo punjenje

```
# app/seed.py
result = await session.execute(
   select(Club).where(Club.name == DEMO_CLUB_NAME)
)
club = result.scalar_one_or_none()
if club is None:
   club = Club(name=DEMO_CLUB_NAME, city=DEMO_CLUB_CITY)
   session.add(club)
   await session.flush()
```

- **Provjera prije upisa** čini postupak ponovljivim: drugo pokretanje ne stvara drugi klub i ne pada na ograničenju `UNIQUE`.
- **`flush`** šalje upis bazi prije potvrde transakcije, pa klub dobije `id` koji korisniku treba kao strani ključ.

Naredba je `python -m app.seed`, i to nakon `alembic upgrade head`: punjenje upisuje retke u tablice koje migracija tek stvara.

Sažetak

- Relacijska baza odabrana je jer domena ima nepromjenjiv oblik zapisa, veze među njima i pravila koja se provode nad svakim upisom.
- Tablica se izvodi iz pojma koji u domeni traje. Redak se određuje vrijednošću bez značenja, jer se podatci iz domene mijenjaju, a veze se opisuju parom primarnog i stranog ključa.
- Ograničenje u shemi vrijedi za svaki upis. Provjera u kodu ne obuhvaća dva istodobna zahtjeva, jer se stanje može promijeniti između provjere i upisa.
- ORM preslikava klasu u tablicu i sastavlja SQL, ali ga ne skriva. Model baze i shema API-ja ostaju razdvojeni.
- Migracija je zapis jedne promjene sheme u repozitoriju. Čita se prije pokretanja, jer `downgrade` ne vraća obrisane podatke.